

Name/NetID: Nathan Kariuki / nkari011

Session Date: May 18, 2026 (1-2 hours)

Objective: Verify all 6 variants with the answer files and add worklist size logging.

Attempts made: Added verification code to main.cpp, the verify functions loop over all vertices and count mismatches, printing the first few if any are found. Ran all 6 variants and checked against the reference files.

Also added a round counter and cout inside each sparse Run() loop to print the frontier size per round. Just one line per round:

```
cout << "BFS sparse round " << round++ << " |F|=" << cur.size() << endl;
```

Did the same for SSSP sparse and CC sparse.

Solutions/Workarounds: All 6 variants passed on the first run. No fixes needed for correctness at this point. The worklist logging just required adding a round variable and a print at the top of the while loop.

Commits: verify: confirm all 6 variants match answer files; exp: add worklist size logging per round

Experiments/Analysis: BFS sparse worklist sizes on LiveJournal:

round 0: 1, round 1: 46, round 2: 2155, round 3: 101386, round 4: 1174905, round 5: 2166715, round 6: 811128, round 7: 122063, round 8: 17877 ... down to 4 by round 14

The frontier grows fast, peaks around round 5 at 2.1M vertices, then shrinks quickly. BFS on a social graph converges in just 14 rounds total.

SSSP sparse ran for 41 rounds and the frontier stayed large through most of them, peaking around 3.4M in rounds 5-7 before slowly tapering off. Much slower to converge than BFS because SSSP vertices can be re-relaxed multiple times.

CC sparse ran for only 7 rounds but round 0 had the full 4.8M vertices since all vertices start active. By round 2 it was down to 311k and dropped fast after that.

The sparse worklist is most efficient when the frontier is small relative to n . For BFS on LiveJournal the early and late rounds benefit a lot but the middle rounds where $|F|$ approaches 2M don't get much out of it. For CC the benefit is limited since round 0 is always dense by construction. SSSP has the most rounds but also keeps a large frontier throughout so the savings are modest.

Learning outcome: Printing the worklist size per round is actually useful beyond just satisfying the spec requirement. It shows you exactly where the algorithm spends its time and which rounds are dense vs sparse. For BFS the frontier profile is a nice bell curve. SSSP is much flatter which explains why it takes so much longer even with fewer vertices to process per round.